

Software Testing

Prof. Dr. Raphael Herding

Software Testing

- Testing is supposed to show that a program does what it is supposed to do.
- Uncover bugs before it goes into production.
- In the test, artificial data is mostly used.
- Results are checked for errors, anomalies or non-functional properties.
- **You can detect the presence of errors, but NOT their absence.**
- No software can be tested for complete accuracy.

Software Testing - Goals

- **Validation tests: Developers** and customers show that the software requirements are met.
 - There must be at least one test for each requirement in the requirements document.
 - There must be tests for all system functions as well as for combinations of these.
- **Defect Testing:** Uncovering situations where the behavior of the software is incorrect or non-compliant with specifications.
 - The aim is to detect unwanted system behavior such as system crashes, unwanted interactions with other systems, erroneous calculations, and data corruption.

Software Testing - Static and Dynamic Tests

- **Software inspections (static verification)** - static system analysis to detect problems.
 - Program is not running
 - Code reviews to find anomalies and bugs early
 - Can be applied to different levels of abstraction (code, architecture, scripts, ...)
 - Can be used to improve quality
- **Software Testing (Dynamic Verification)** - Deals with testing and observing system behavior.
 - Program runs with test data
- Both are necessary. They complement each other.

Software Testing - Classification

- **Static procedure** – program is not executed
- **Dynamic Procedures** – Program Runs on Test Data
- **Analytic Techniques** – Measuring Code Quality
- **Verifying procedures** – proof of system correctness
- **White-box procedure** – "looking" into the program code
- **Black-Box Method** – Interface Testing

Software Testing Classification

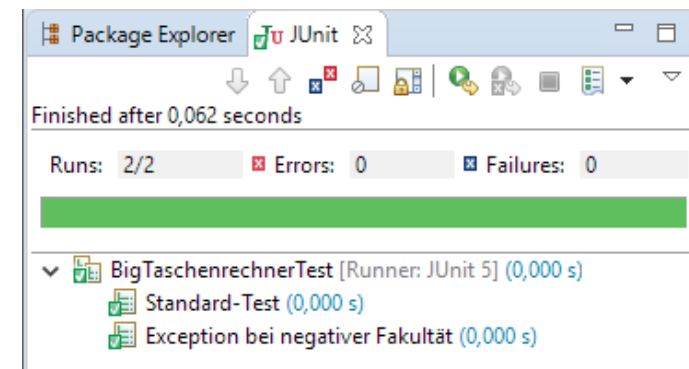
- **Static procedure** – program is not executed
 - Code analysis tools are often used (SonarCube, Checkstyle, ...)
- **Dynamic Procedures**
 - Program runs with test data
 - Based on a systematic selection of test data, as many errors as possible are to be uncovered
- Static and dynamic test methods are often used in practice

Software Testing - Classification

- In practice, unit testing is often used (dynamic testing).
- Supported in Java by JUnit framework.

```
class BigTaschenrechnerTest {  
  
    @Test  
    @DisplayName("Standard-Test")  
    void testFakultaet() {  
        assertEquals(BigInteger.ONE, BigTaschenrechner.fakultaet(0));  
        assertEquals(BigInteger.ONE, BigTaschenrechner.fakultaet(1));  
        assertEquals(BigInteger.valueOf(2), BigTaschenrechner.fakultaet(2));  
        //assertEquals(788657867364790503552363213932185062295135977687173263294742533244359449!  
    }  
  
    @Test  
    @DisplayName("Exception bei negativer Fakultät")  
    void testFakultaetException() {  
        assertThrows(IllegalArgumentException.class, () -> {BigTaschenrechner.fakultaet(-1)});  
    }  
}
```

```
import java.math.BigInteger;  
  
public class BigTaschenrechner {  
    public static BigInteger fakultaet(int zahl) {  
        if (zahl < 0) throw new IllegalArgumentException();  
        if (zahl == 0) return BigInteger.ONE;  
        BigInteger ergebnis = BigInteger.ONE;  
        for (int i=1;i<=zahl;i++) {  
            ergebnis = ergebnis.multiply(BigInteger.valueOf(i));  
        }  
        return ergebnis;  
    }  
}
```



Software Testing - Classification

- **Analytical methods**

- Try to make statements about quality based on a purely syntactic analysis
- Procedures can often be automated (metrics, anomaly analysis) or are carried out manually (review, walkthrough)
- **Automated procedures** - provide cost-effective initial indications of problem areas
- **Reviews** - can always be used in all phases of software development and help to detect gross errors and problems at an early stage

- **Verification procedures**

- Prove the *formal correctness* of analysis, design or program documents or important properties of software documents
- Elaborate and complex
- Not automatable
- Used in safety-critical areas

Software Testing - Classification

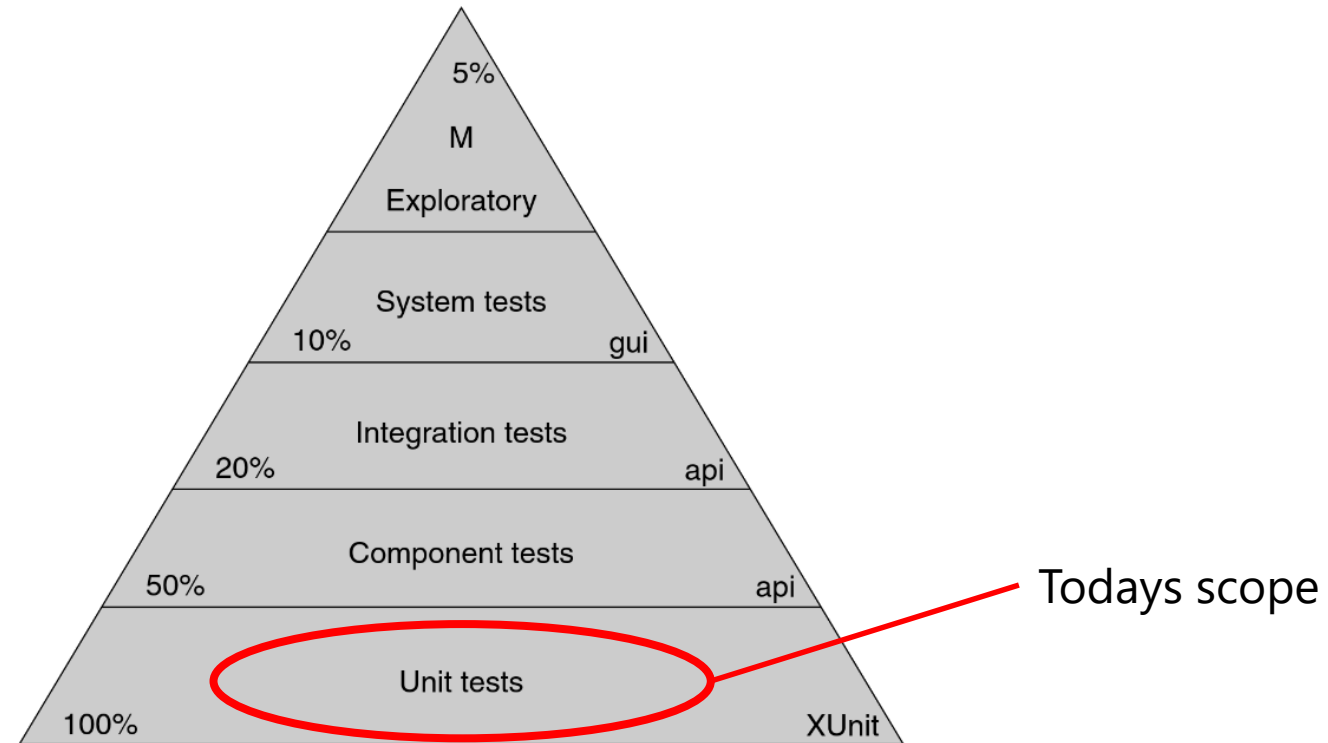
- **White-box process**

- Require knowledge of the internal structures of the software system.
- Use this structural knowledge in testing.

- **Black Box Procedure**

- Pure external view
- Internal structure is unimportant
- Only open interfaces are considered
- Interface-Testing

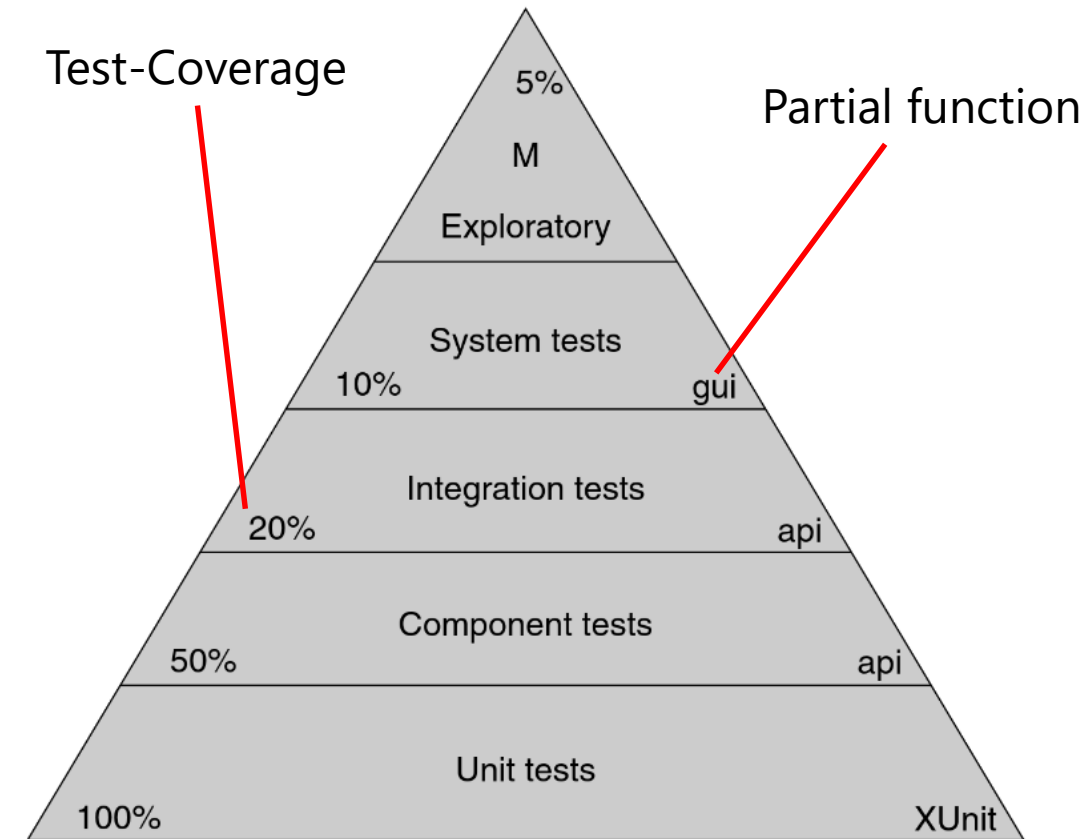
Software Testing - Testpyramide



- Multiple levels of testing are required to ensure good test coverage
- Goal: the highest possible test coverage to increase software quality
- There are different pyramids depending on the application (here Robert C. Martin)
- Automate testing whenever possible

Software Testing - Testpyramide

- **Unit tests** – small unit in program code (single class or method/s)
- **Component tests** – Compilation of several units (logical class association)
- **Integration tests** – software fully created and running. Integration of all components with each other is being tested. End-to-end tests, security tests and performance tests are carried out here



Software Testing - Testpyramide

- **System Testing** – Testing the "installed" overall system. Testing the surface end-to-end.
- **Exploratory** – Manual testing after the system has been installed. Final quality assurance measures. "How does the product feel".

